FEAT-ZD-48600 · DES-01 Platform Architecture Update [1] · Platform architecture change (when flagged)

Design agent · item feat.platform_update · source hobasa-design-agent/outputs/FEAT-ZD-48600/checks/DES-01_feat.platform_update.md · approved by automated checks and independent judge

feat.platform_update: FEAT-ZD-48600 (DES-01 check record)

Step DES-01 Platform Architecture Update, attempt 1, run in **check mode**. The step's condition, "TRD flags a platform-level change", is not met in this run (record:intake). This record states what was checked, the evidence, the conclusion and what would trigger a platform architecture update. No architecture document, code, configuration, test or environment was changed.

Field	Value	Source
Trace ID	FEAT-ZD-48600	record:brief; feat.trd
Step / owner	DES-01 Platform Architecture Update / Architect	record:brief
Mode	Check (condition not met: TRD flags a platform-level change)	record:intake
Inputs read	feat.trd (approved at REQ-07), org.architecture (feature override)	record:brief
Shared memory read	D-2026-031, D-2026-044, D-2026-050; L-007, L-011	record:memory

1. What a platform-level change means here

org.architecture defines the platform as these elements. A platform-level change is a change to any of them (agent judgement on the scope of "platform level", taken directly from what org.architecture states):

#	Platform element	Current statement	Source
P-1	Topology	Four repositories in one flow: ap_pipeline → ap_forecast / ap_aging → ap_portal	org.architecture
P-2	Runtimes	ap_pipeline, ap_forecast, ap_aging on Python 3.9; ap_portal on Node.js 22 CommonJS	org.architecture
P-3	Repository roles	ap_pipeline produces Invoice records; ap_forecast predicts payment delay; ap_aging computes aging; ap_portal renders HTML from JSON	org.architecture
P-4	Inter-service data rule	JSON between services carries amounts as strings with two decimals and dates as ISO strings	org.architecture
P-5	Compatibility rule	"Keep every existing signature; add new functions."	org.architecture
P-6	Security and tenancy boundary	No finance posting, authentication, tenancy or payment execution in scope of feature work at tier T2	feat.trd (S-1)

2. Checks and evidence

#	Check	Evidence in the TRD	Result
CK-1	Does the feature add, remove or re-order a repository or flow stage (P-1)?	All components C-1 to C-10 sit in the existing four repositories plus a test harness (C-10). The end-to-end wiring IF-8 follows the existing order: pipeline → forecast → aging → portal.	No change
CK-2	Does it change a runtime or language version (P-2)?	F-15 restates Python 3.9 and Node.js 22 CommonJS and requires new Python code to stay within 3.9 syntax.	No change
CK-3	Does it move a responsibility between repositories (P-3)?	Each new function stays in the repository that owns its concern: `delays_by_vendor` in ap_pipeline (C-2), `vendor_risk` in ap_forecast (C-4), `overdue_with_risk` and `overdue_rows_to_json` in ap_aging (C-6, C-7), `renderOverdueRisk` in ap_portal (C-8). Lane routing is unchanged (TRD header, org.routing_map).	No change
CK-4	Does it change the inter-service JSON rule (P-4)?	The new contract CT-1 (IF-6) carries `amount` as a two-decimal string and has no date field; it states that any date added later must be an ISO string. CT-1 is a feature-level contract that **conforms** to P-4; it does not alter the rule.	No change (conforms)
CK-5	Does it break the compatibility rule (P-5)?	TRD section 9: only CR-1 (`to_model` behaviour) and CR-4 (`risk=None` as last parameter) touch existing functions, both additive, signatures unchanged. `aging_buckets`, `days_past_due`, `bucket_for`, `formatAmount`, `renderAgingSummary` unchanged (IF-5.8, C-9).	No change (rule kept)
CK-6	Does it add storage, a schema migration, a service, a transport or a network dependency?	TRD section 3: "No storage, schema migration or new persistent data. Every new function is in-memory and pure." IF-6 / R-7: no transport layer exists between ap_aging and ap_portal; C-7 adds only a serialiser function in ap_aging. `vendor_risk` makes no LLM or network call (IF-3.2).	No change
CK-7	Does it touch the security or tenancy boundary (P-6)?	S-1: no finance posting, authentication, tenancy or payment execution added or changed; tier T2. Section 1 "Not affected" also lists the LLM provider or model and aging export.	No change

#	Check	Evidence in the TRD	Result
CK-8	Does it change the LLM provider, model or external data flow at platform level?	IF-4.4: provider, model and `max_tokens` still come from `config` / `models.json`, unchanged. S-5: the explanation prompt additionally carries the vendor's risk label to the existing provider (`acme-llm`); this is a content change within an existing data flow, open for the Architect as TQ-8.	No platform change; open data-flow question (TQ-8)
CK-9	Does the vendor feed change require a platform change?	D-1: feed v3 adds an optional `paid_date` column; schedule unchanged (daily 02:00 Asia/Kolkata). IF-1.1 keeps v2 feeds loading. This is a customer data mapping inside ap_pipeline, not a platform element in org.architecture.	No change
CK-10	Do the shared decisions need a platform-level rule change?	New code follows D-2026-031 (IF-5.2), D-2026-044 (IF-5.5, IF-6, IF-7.5) and D-2026-050 (IF-5.1) (TRD section 9). Existing-code conflicts F-10 (`bucket_for(30)`) and F-12 (`formatAmount` through float) are raised as separate items TQ-1 and TQ-2, not as platform changes.	No change

3. Conclusion

No platform-level architecture change is needed for FEAT-ZD-48600 now. All six platform elements P-1 to P-6 are kept. The feature is a set of additive functions inside the existing four repositories, plus a feature-level JSON contract (CT-1) that conforms to the existing inter-service rule. org.architecture is therefore not updated by this step, and the action "update the platform architectural level document" is not taken.

Note for later design steps: CT-1 (IF-6) is recorded as a feature contract in the TRD. Whether it should also be registered in the platform architecture as a named inter-service contract is an Architect choice; it does not change any platform rule (non-blocking question Q-1 below).

4. What would trigger the action (re-run DES-01 as a real update)

#	Trigger	Linked open item	Source
TR-1	The Architect decides that a transport, API or service carries CT-1 between ap_aging and ap_portal (new platform component or inter-service channel)	TQ-3, R-7	feat.trd (IF-6, C-7, R-7, TQ-3)
TR-2	`bucket_for` or `formatAmount` is fixed as a platform-wide rule change (for example, a bucket boundary or amount-format rule for all consumers) rather than a local fix	TQ-1, TQ-2	feat.trd (F-10, F-12, R-1, R-2)
TR-3	Any new function gains storage, a schema migration, or a network / LLM call	—	feat.trd (section 3, IF-3.2)

#	Trigger	Linked open item	Source
TR-4	The LLM provider or model changes, or the Architect rejects sending the risk label to the provider and a new data-handling boundary is needed	TQ-8, PQ-5, R-3	feat.trd (S-5, IF-4.4, R-3)
TR-5	A runtime version changes, or an existing signature must change (break of P-5)	—	feat.trd (F-15, section 9); org.architecture
TR-6	The end-to-end harness (C-10, IF-8) becomes a deployed production component instead of a test	R-9	feat.trd (C-10, IF-8, R-9)

5. Open questions (none blocking)

ID	Question	Why it matters
Q-1	Should CT-1 be registered in org.architecture as a named inter-service contract, or stay a feature-level contract in feat.trd?	Discoverability for other features; no platform rule changes either way
Q-2	Carried: TQ-3 (owner of serialisation and transport between ap_aging and ap_portal) and TQ-8 (risk label sent to the external provider). Either answer may hit TR-1 or TR-4.	Could turn this check into a real platform update